DOC-SMCTA-ARQ-002 · VERSIÓN 2.1.0

Diseño del Modelo de Datos Relacional e Invariantes Contables

Esquema Físico PostgreSQL 15+, Índices Parciales de Exclusión Mutua y Auditoría Append-Only

Proyecto / Sistema	**Arquitectura de Sistemas Transaccionales Multi-Tenant**
Fase / Área Normativa	**03 — Arquitectura y Diseño de Software**
Autoría y Gobernanza	**Equipo de Ingeniería de Software — Proyecto SMCTA**
Fecha de Emisión	**Noviembre 2026**

Resumen Ejecutivo

Este documento establece la especificación física y formal del modelo relacional de datos del **Sistema de Mercado Comportamental y Trading de Activos (SMCTA v2)** sobre **PostgreSQL 15+** (en producción) y **PGlite WASM** (en desarrollo local y ejecución de pruebas deterministas). A diferencia de los esquemas CRUD convencionales donde las reglas de negocio residen únicamente en memoria de aplicación, este modelo traslada los invariantes financieros críticos al motor relacional mediante **restricciones de dominio (`CHECK`)**, **índices únicos parciales (`CREATE UNIQUE INDEX ... WHERE`)** contra doble gasto y **disparadores PL/pgSQL (`BEFORE UPDATE OR DELETE`)** que imponen inmutabilidad estricta sobre el libro mayor de custodia (*Escrow*).

Índice General de Contenidos

Resumen Ejecutivo

1. Catálogo de Entidades y Estrategia de Aislamiento Multi-Tenant

2. Diagrama Entidad-Relación Físico (ERD Vectorial)

3. Diccionario Detallado de Atributos y Dominios de Precisión

- 3.1. Tabla `tenants` (Configuración Paramétrica B2B y Branding)
- 3.2. Tabla `coupons` (Activos Digitales y Versión Criptográfica)
- 3.3. Tablas `p2p_orders` y `escrow_ledgers` (Liquidación y Custodia)

4. Invariantes de Concurrencia y Seguridad en el Motor SQL

5. Especificación DDL Canónica (`schema.sql`)

6. Parametrización Operativa por Entorno (Adaptador Hexagonal `Database`)

1. Catálogo de Entidades y Estrategia de Aislamiento Multi-Tenant

El esquema implementa el patrón *Shared Database, Shared Schema* con particionamiento lógico obligatorio por columna discriminadora `tenant_id`. Ninguna entidad operativa existe fuera del ámbito de un comercio emisor (*Tenant* B2B).

Entidad Física	Rol Arquitectónico y Semántica de Dominio	Clave Primaria	Aislamiento y Cardinalidad
`tenants`	Raíz de configuración B2B: parámetros de comisiones (*take-rates*), bandas de precio (*Price Collar*), cupo diario de reventa, ventana de cierre pre-evento, secreto criptográfico `qr_secret` y tokens de diseño *White-Label*.	`tenant_id` (`UUID`)	Entidad raíz maestra (`1:N` hacia el resto del esquema; `ON DELETE RESTRICT`).
`users`	Sujetos transaccionales (consumidores primarios y operadores del mercado secundario P2P) registrados bajo un recinto comercial específico.	`user_id` (`UUID`)	`FK tenant_id` + restricción compuesta `uq_users_tenant_email (tenant_id, email)` .
`coupons`	Unidad digital de derecho de consumo respaldada al 100% por su valor nominal en custodia. Almacena titular actual, estado del autómata finito y contador monótono `token_version` .	`coupon_id` (`UUID`)	`FK tenant_id` + índice compuesto `idx_coupons_owner (tenant_id, current_owner_id)` .
`p2p_orders`	Libro de órdenes de oferta secundaria (*Order Book*). Registra precio demandado (`asking_price`), estado de ejecución y desglose exacto de comisiones al momento del *match*.	`order_id` (`UUID`)	`FK tenant_id` , `FK coupon_id` + índice parcial `uq_orders_open_coupon` .
`escrow_ledgers`	Libro mayor contable de solo adición (*append-only ledger*) que audita ingresos a custodia, liquidaciones de comisiones P2P, liberaciones por canje en TPV y reembolsos por expiración.	`ledger_id` (`UUID`)	`FK tenant_id` , `FK coupon_id` + trigger inmutable `trg_escrow_ledgers_immutable` .
`user_events`	Bandeja persistente de notificaciones transaccionales para reconciliación asíncrona del canal reactivo *Server-Sent Events* (SSE).	`event_id` (`UUID`)	`FK tenant_id` + índice cronológico inverso `idx_events_user (tenant_id, user_id, created_at DESC)` .

2. Diagrama Entidad-Relación Físico (ERD Vectorial)

La siguiente figura representa la topología relacional exacta desplegada en `apps/backend/src/db/schema.sql` , incluyendo tipos enumerados nativos y restricciones de integridad referencial.

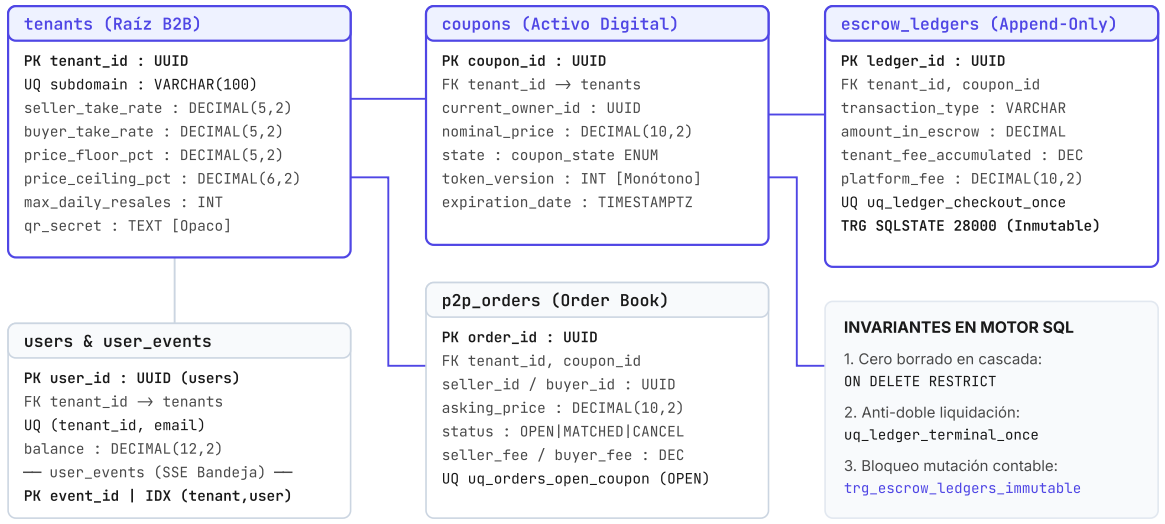


Figura 1. Esquema Entidad-Relación físico de `apps/backend/src/db/schema.sql` (PostgreSQL 15 / PGlite WASM), destacando las seis tablas productivas, los índices únicos parciales y el disparador de inmutabilidad contable.

> *Invariante de Integridad Referencial: Todas las claves foráneas hacia `tenants(tenant_id)` y `coupons(coupon_id)` declaran `ON DELETE RESTRICT`. Queda prohibido el borrado físico en cascada (`CASCADE`) en producción para preservar la trazabilidad forense de cada asiento contable histórico.*

3. Diccionario Detallado de Atributos y Dominios de Precisión

Todos los importes monetarios y tasas porcentuales utilizan exclusivamente el tipo exacto `DECIMAL(p, s)` de base 10 en PostgreSQL, operando en correspondencia biunívoca con `decimal.js` (`ROUND_HALF_EVEN`) en la capa de dominio para erradicar errores de representación binaria IEEE 754.

3.1. Tabla `tenants` (Configuración Paramétrica B2B y Branding)

Columna	Tipo SQL	Nulabilidad / Default	Restricción e Invariante de Negocio
`tenant_id`	`UUID`	`PK DEFAULT gen_random_uuid()`	Identificador inmutable del comercio o evento.

Columna	Tipo SQL	Nulabilidad / Default	Restricción e Invariante de Negocio
`company_name`	`VARCHAR(150)`	`NOT NULL`	Razón social o marca visible en cabecera *White-Label*.
`subdomain`	`VARCHAR(100)`	`UNIQUE NOT NULL`	Clave de resolución en `tenantResolver` (ej. `festival`, `hotel`, `cafeteria`).
`seller_take_rate`	`DECIMAL(5,2)`	`NOT NULL DEFAULT 3.50`	Comisión retenida al vendedor P2P (`ck_tenants_rates ⩾ 0`).
`buyer_take_rate`	`DECIMAL(5,2)`	`NOT NULL DEFAULT 2.50`	Recargo operativo aplicado al comprador P2P (`ck_tenants_rates ⩾ 0`).
`price_floor_pct`	`DECIMAL(5,2)`	`NOT NULL DEFAULT 50.00`	Límite inferior del *Price Collar* respecto al valor nominal.
`price_ceiling_pct`	`DECIMAL(6,2)`	`NOT NULL DEFAULT 200.00`	Límite superior del *Price Collar* (`ck_tenants_collar: floor ⩽ ceiling`).
`max_daily_resales_per_user`	`INT`	`NOT NULL DEFAULT 5`	Límite diario anti-acaparamiento por vendedor (RN-04).
`closure_hours_before_event`	`INT`	`NOT NULL DEFAULT 2`	Ventana previa a `expiration_date` que congela el mercado secundario.
`platform_revenue_share_pct`	`DECIMAL(5,2)`	`NOT NULL DEFAULT 30.00`	Participación de la plataforma SaaS sobre las comisiones brutas P2P.
`qr_secret`	`TEXT`	`NOT NULL`	Secreto criptográfico por *tenant* encapsulado en `QrSecret` (nunca serializado).
`theme_*`	`VARCHAR / TEXT`	`NOT NULL`	Cinco atributos de identidad visual (`primary`, `accent`, `surface`, `font`, `logo`).

3.2. Tabla `coupons` (Activos Digitales y Versión Criptográfica)

Columna	Tipo SQL	Nulabilidad / Default	Restricción e Invariante de Negocio
`coupon_id`	`UUID`	`PK DEFAULT gen_random_uuid()`	Identificador único global del derecho de consumo emitido.
`tenant_id`	`UUID`	`NOT NULL FK`	Pertenencia estricta al recinto emisor; impide canje cruzado entre marcas.
`current_owner_id`	`UUID`	`NOT NULL`	Titular legítimo actual. Sin `FK` rígida a `users` para soportar cuentas efímeras de carga.

Columna	Tipo SQL	Nulabilidad / Default	Restricción e Invariante de Negocio
`nominal_price`	`DECIMAL(10,2)`	`NOT NULL`	Capital exacto depositado en custodia primaria (`ck_coupons_nominal_pos > 0`).
`state`	`coupon_state`	`NOT NULL DEFAULT 'EMITIDO'`	Enumerado: `EMITIDO` , `EN_WALLET` , `PUBLICADO_P2P` , `CANJEADO` , `VENCIDO` .
`product_name`	`VARCHAR(160)`	`NOT NULL DEFAULT ''`	Denominación comercial del producto o pase asociado al cupón.
`token_version`	`INT`	`NOT NULL DEFAULT 0`	Contador monótono incrementado en cada cambio de estado/dueño (defensa anti-boomerang QR).
`expiration_date`	`TIMESTAMPTZ`	`NOT NULL`	Instante límite de validez comercial y referencia de cierre del mercado P2P.

3.3. Tablas `p2p_orders` y `escrow_ledgers` (Liquidación y Custodia)

Tabla	Columnas Críticas	Tipos SQL	Garantía Estructural
`p2p_orders`	`asking_price` , `seller_fee_amount` , `buyer_fee_amount` , `net_seller_proceeds` , `total_buyer_price`	`DECIMAL(10,2)`	Persiste la fotografía contable exacta en el instante de `MATCHED` (`matched_at TIMESTAMPTZ`), inmune a cambios futuros de tasas en `tenants` .
`escrow_ledgers`	`transaction_type` , `amount_in_escrow` , `tenant_fee_accumulated` , `platform_fee` , `related_order_id`	`VARCHAR(50)` , `DECIMAL(10,2)` , `UUID`	`CHECK` restringido a `('CHECKOUT', 'P2P_SPLIT', 'REDEEM_RELEASE', 'REFUND')` . Invariante: suma algebraica de `amount_in_escrow` de cupones cerrados = `0.00` .

4. Invariantes de Concurrencia y Seguridad en el Motor SQL

El esquema implementa tres mecanismos formales que blindan la consistencia incluso ante fallos de red en la capa de bloqueos distribuidos:

1. **Unicidad Parcial de Oferta Abierta (`uq_orders_open_coupon`):** Impide a nivel de índice B-Tree que un mismo `coupon_id` posea simultáneamente dos órdenes con `status = 'OPEN'` .
2. **Idempotencia Contable de Custodia (`uq_ledger_checkout_once` y `uq_ledger_terminal_once`):** Garantiza matemáticamente que ningún cupón pueda

fondear dos veces la cuenta Escrow en emisión primaria ni ejecutar una doble liberación terminal (`REDEEM_RELEASE` o `REFUND`).

3. **Inmutabilidad Criptográfica-Operativa (`trg_escrow_ledgers_immutable`):** Cualquier sentencia `UPDATE` o `DELETE` ejecutada sobre `escrow_ledgers` —incluso desde una sesión administrativa ordinaria— aborta de inmediato con `SQLSTATE 28000` (*invalid_authorization_specification*).

---

5. Especificación DDL Canónica (`schema.sql`)

SQL

```
DO $$ BEGIN
    CREATE TYPE coupon_state AS ENUM ('EMITIDO', 'EN_WALLET',
'PUBLICADO_P2P', 'CANJEADO', 'VENCIDO');
EXCEPTION WHEN duplicate_object THEN NULL; END $$;

DO $$ BEGIN
    CREATE TYPE order_side AS ENUM ('BUY', 'SELL');
EXCEPTION WHEN duplicate_object THEN NULL; END $$;

DO $$ BEGIN
    CREATE TYPE order_status AS ENUM ('OPEN', 'MATCHED', 'CANCELLED');
EXCEPTION WHEN duplicate_object THEN NULL; END $$;

CREATE TABLE IF NOT EXISTS tenants (
    tenant_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    company_name VARCHAR(150) NOT NULL,
    subdomain VARCHAR(100) UNIQUE NOT NULL,
    seller_take_rate DECIMAL(5,2) NOT NULL DEFAULT 3.50,
    buyer_take_rate DECIMAL(5,2) NOT NULL DEFAULT 2.50,
    price_floor_pct DECIMAL(5,2) NOT NULL DEFAULT 50.00,
    price_ceiling_pct DECIMAL(6,2) NOT NULL DEFAULT 200.00,
    max_daily_resales_per_user INT NOT NULL DEFAULT 5,
    closure_hours_before_event INT NOT NULL DEFAULT 2,
    platform_revenue_share_pct DECIMAL(5,2) NOT NULL DEFAULT 30.00,
    qr_secret TEXT NOT NULL,
    theme_primary_color VARCHAR(9) NOT NULL DEFAULT '#1A365D',
    theme_accent_color VARCHAR(9) NOT NULL DEFAULT '#3182CE',
    theme_surface_color VARCHAR(9) NOT NULL DEFAULT '#FFFFFF',
    theme_font_family VARCHAR(120) NOT NULL DEFAULT 'Inter, sans-serif',
    theme_logo_url TEXT NOT NULL DEFAULT '',
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT ck_tenants_collar CHECK (price_floor_pct ⩽
price_ceiling_pct),
    CONSTRAINT ck_tenants_rates CHECK (seller_take_rate ⩾ 0 AND
buyer_take_rate ⩾ 0)
);

CREATE TABLE IF NOT EXISTS users (
```

```
    user_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    display_name VARCHAR(120) NOT NULL,
    email VARCHAR(160) NOT NULL,
    balance DECIMAL(12,2) NOT NULL DEFAULT 0.00,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_users_tenant FOREIGN KEY (tenant_id) REFERENCES
tenants(tenant_id) ON DELETE RESTRICT,
    CONSTRAINT uq_users_tenant_email UNIQUE (tenant_id, email)
);

CREATE TABLE IF NOT EXISTS coupons (
    coupon_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    current_owner_id UUID NOT NULL,
    nominal_price DECIMAL(10,2) NOT NULL,
    state coupon_state NOT NULL DEFAULT 'EMITIDO',
    qr_encrypted_token TEXT NOT NULL DEFAULT '',
    expiration_date TIMESTAMP WITH TIME ZONE NOT NULL,
    product_name VARCHAR(160) NOT NULL DEFAULT '',
    token_version INT NOT NULL DEFAULT 0,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_coupons_tenant FOREIGN KEY (tenant_id) REFERENCES
tenants(tenant_id) ON DELETE RESTRICT,
    CONSTRAINT ck_coupons_nominal_pos CHECK (nominal_price > 0)
);

CREATE TABLE IF NOT EXISTS p2p_orders (
    order_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    coupon_id UUID NOT NULL,
    seller_id UUID NOT NULL,
    asking_price DECIMAL(10,2) NOT NULL,
    side order_side NOT NULL DEFAULT 'SELL',
    status order_status NOT NULL DEFAULT 'OPEN',
    buyer_id UUID,
    seller_fee_amount DECIMAL(10,2),
    buyer_fee_amount DECIMAL(10,2),
    net_seller_proceeds DECIMAL(10,2),
    total_buyer_price DECIMAL(10,2),
    matched_at TIMESTAMP WITH TIME ZONE,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_orders_tenant FOREIGN KEY (tenant_id) REFERENCES
tenants(tenant_id) ON DELETE RESTRICT,
    CONSTRAINT fk_orders_coupon FOREIGN KEY (coupon_id) REFERENCES
coupons(coupon_id) ON DELETE RESTRICT,
    CONSTRAINT ck_orders_asking_pos CHECK (asking_price > 0)
);

CREATE UNIQUE INDEX IF NOT EXISTS uq_orders_open_coupon
    ON p2p_orders(coupon_id) WHERE status = 'OPEN';

CREATE TABLE IF NOT EXISTS escrow_ledgers (
    ledger_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
    tenant_id UUID NOT NULL,
    coupon_id UUID NOT NULL,
    transaction_type VARCHAR(50) NOT NULL,
    amount_in_escrow DECIMAL(10,2) NOT NULL,
    tenant_fee_accumulated DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    platform_fee DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    related_order_id UUID,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_ledger_tenant FOREIGN KEY (tenant_id) REFERENCES
tenants(tenant_id) ON DELETE RESTRICT,
    CONSTRAINT fk_ledger_coupon FOREIGN KEY (coupon_id) REFERENCES
coupons(coupon_id) ON DELETE RESTRICT,
    CONSTRAINT ck_ledger_tx_type CHECK (
        transaction_type IN ('CHECKOUT', 'P2P_SPLIT', 'REDEEM_RELEASE',
'REFUND')
    )
);

CREATE UNIQUE INDEX IF NOT EXISTS uq_ledger_checkout_once
    ON escrow_ledgers(coupon_id) WHERE transaction_type = 'CHECKOUT';
CREATE UNIQUE INDEX IF NOT EXISTS uq_ledger_terminal_once
    ON escrow_ledgers(coupon_id) WHERE transaction_type IN
('REDEEM_RELEASE', 'REFUND');

CREATE OR REPLACE FUNCTION trg_prevent_ledger_mutation()
RETURNS trigger AS $$
BEGIN
    RAISE EXCEPTION 'Operación % no permitida sobre tabla append-only %
(SQLSTATE: 28000)', TG_OP, TG_TABLE_NAME
        USING ERRCODE = '28000';
END;
$$ LANGUAGE plpgsql;

DROP TRIGGER IF EXISTS trg_escrow_ledgers_immutable ON escrow_ledgers;
CREATE TRIGGER trg_escrow_ledgers_immutable
BEFORE UPDATE OR DELETE ON escrow_ledgers
FOR EACH ROW EXECUTE FUNCTION trg_prevent_ledger_mutation();

CREATE TABLE IF NOT EXISTS user_events (
    event_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    tenant_id UUID NOT NULL,
    user_id UUID NOT NULL,
    type VARCHAR(40) NOT NULL,
    title VARCHAR(200) NOT NULL,
    amount DECIMAL(10,2) NOT NULL DEFAULT 0.00,
    related_coupon_id UUID,
    related_order_id UUID,
    is_read BOOLEAN NOT NULL DEFAULT FALSE,
    created_at TIMESTAMP WITH TIME ZONE NOT NULL DEFAULT CURRENT_TIMESTAMP,
    CONSTRAINT fk_events_tenant FOREIGN KEY (tenant_id) REFERENCES
tenants(tenant_id) ON DELETE RESTRICT
);
```

6. Parametrización Operativa por Entorno (Adaptador Hexagonal `Database`)

Parámetro Arquitectónico	Entorno Local / CI (`PGliteDatabase`)	Entorno Productivo (`PgDatabase`)
Motor de Persistencia	`@electric-sql/pglite` v0.2.17 (PostgreSQL compilado a WASM)	PostgreSQL 15.4+ sobre contenedor dedicado / RDS
Nivel de Aislamiento Transaccional	`SERIALIZABLE` sobre conexión única con cola FIFO (`mutexChain`)	`BEGIN ISOLATION LEVEL SERIALIZABLE` sobre pool `pg.Pool` (`max: 20`)
Semilla Multi-Tenant (`seed.ts`)	4 *Tenants* preconfigurados (`cafeteria`, `festival`, `hotel`, `sneakers`)	Provisionamiento idempotente mediante migración controlada
Autenticación de Tokens QR	`HMAC-SHA256` con `qr_secret` aislado por fila en `tenants`	`HMAC-SHA256` con rotación de secretos en gestor KMS/Vault